Make assert() macro user friendly for C and C++

Document #:	P2246R1 (WG21)/ NXXXX (was N2621) (WG14)
Date:	2021-10-05
Project:	Programming Language C++
Audience:	WG21 - Library Evolution and SG22 (C++/C-Liaison), WG14

Reply-to:	Peter Sommerlad <peter.cpp@sommerlad.ch>

Contents

§ 1 Introduction

The `assert()` macro, being a macro, is not very beginner friendly in C++, because the preprocessor only uses parenthesis for pairing and none of the other structuring syntax of C++, such as template angle brackets, or curly braces. This makes it user unfriendly in a C++ context, requiring an extra pair of parentheses, if the expression used, incorporates a comma.

Shafik Yaghmour presented the following C++ code in one of his Twitter quizzes `tweet` demonstrating the weakness.

```
#include <cassert>
#include <type_traits>

using Int=int;
```

```
void f() {
 assert(std::is_same<int,Int>::value); // a surprisig compile error
}
```

One of the twitter responses (by @_Static_assert) to the tweet mentioned above, even provided a definition of the assert macro that actually is a primitive implementation of what I propose in this paper:

```
#define assert(...) ((__VA_ARGS__)?(void)0:std::abort())
```

In C one needs to be a bit more sophisticated to trigger such a compile error, but nevertheless the C syntax allows for such expression that include commas that are not protected from the preprocessor by parentheses as given by Shafik's godbolt example

```
#include <assert.h>

void f() {
    assert((int[2]){1,2}[0]); // compile error
    struct A {int x,y;};
    assert((struct A){1,2}.x); // compile error
}
```

The current C standard does not even sanction such a compile error to my knowledge, when `NDEBUG` is not defined, since it specifies the assert macro to be able to take an expression of *scalar type* which the above non-compiling examples with a comma, I think, are (int in both cases). The C++ standard and working paper refer to C's definition of the `assert` macro in that respect.

§ 2 Remedy

This deficit in the single argument macro assert() seems to be very easy to mitigate by providing a `__VA_ARGS__` version of the macro using ellipsis (. . .) as the parameter.

There exist the option to specify the assert macro with an extra name parameter and then the ellipsis. However, I do think this not only complicates its implementation it also complicates its wording, as well as this feature allowing a single argument macro call is not available for C++ versions pre C++20. If the `assert` macro is called without any arguments this will lead to a compile error as it does today. The only difference might be the issued compiler diagnostic

A DIY version can be defined that provides the additional parenthesis needed for the `assert()` macro of today:

```
#define Assert(...) assert((__VA_ARGS__))
```

However, that would be required to be defined and used throughout a project and such is less user friendly than have the standard facility provide such flexibility.

Note: the following *feature* was removed, due to discussions of the R0/N2621 version of this paper. And a mechanism is enforced to detect misuse of the comma operator.

~~In addition the variable argument macro version of assert would allow additional diagnostics text, by using the comma operator, such as in~~

```
    assert((void)"this cannot be true", -0.0);
```

~~which would otherwise also be required to use an extra pair of parentheses.~~

However, such additional diagnostic strings are better spelled using the `&&` conjugation (thanks to Martin Hořeňovský)

```
assert(idx < vec.size() && "idx is out of range");
```

The proposed solution prevents the use of the comma operator on top-level, to avoid accidentally creating always true assertions like the following:

```
    assert(x > 0 , "x was not greater than zero");
```

Those assertions can result from converting a `static_assert` to a regular assert.

§ 3 Impact on existing code

On my Mac I changed the system's `assert.h` header to provide variadic macro versions of the `assert(...)` macro for C++ and C. I implemented a mechanism to prevent unintentional use of the comma operator within the macro's arguments. I compiled various software (including LLVM) on my Mac using that changed system header and did not encounter problems, beyond my own bugs that I made in that change. The latter is, why I know that the adapted header was actually used.

When reading the C specification of the semantics of assert() one could argue that the macro parameter should already have been variadic, because even in C one can form a scalar expression with a comma that doesn't require balanced parentheses. So WG14 and WG21 might even consider to apply this change as a backward defect fix to previous revisions of the standards.

§ 4 Potential liabilities of the proposed change

While sharing a preview of this document and during review of the initial revision in the various online study and work groups addressed, I got several people commenting on it. While some were in favor, there were raised some potential issues that I'd like to share paraphrased below.

There were liabilities that I do not list, because they are already addressed by the initial revision of this paper

— breaking compatibility with C: This is not planned, see this paper.

— wording only addresses the NDEBUG case (for C++): I provide now more context, the wording changes required for C++ definitely only relevant with the NDEBUG case. However, if desired, we could split adoption of C and C++ in parallel, but providing full assert specification in C++. This is a much more elaborate task that I refrained from at the moment.

§ 4.1 NDEBUG will allow `assert()` without any arguments

As specified at the moment, the NDEBUG version of `assert(...)` will swallow any macro arguments, even if there is none. I believe this is a small price to pay, since any invalid code that matches a macro argument is allowed today anyway if NDEBUG is set. A more sophisticated specification could use the more-than-one argument variadic macro syntax mentioned below.

§ 4.2 "Contracts will make the 50 year old assert macro obsolete and to not suffer from the macro parsing issue."

While I appreciate the notion of contracts, I and others think it is worthwhile to make `assert()` more beginner friendly, since professional code bases will have their own versions of precondition checking stuff anyway. While beginners can be shown a universally available feature that is identical to C and could even be ported to older revisions of the standards without breaking existing code.

§ 4.3 *"Using the comma operator can be misapplied to an always true assert, if its arguments are formed as for `static_assert(condition,reason)`. This will make wrong code compile that today doesn't."*

This problem is prevented by my implementation, unless `NDEBUG` is defined, by defining an identity function taking a single argument. I do not think we need to update the specification with that respect.

§ 4.4 "Teachability is not improved, because we can teach use extra parentheses today."

I have a lot of experience in teaching C++ and i believe that using those extra parenthesis is teachable when interacting with students having such a problem, but the surprise and the time it takes to remember this remedy when it hits, is worth the effort to make it more user friendly. Especially, since assert() tends to be used as a unit testing framework substitute and thus in C++ the use of templates or initializer lists happens frequently, at least in tests I write.

During discussions in WG21/SG22 and WG14 the following issues were raised:

§ 4.5 Will changing `static_assert(condition,"reason")` to `assert(condition,"reason")` compile and silently make the assert never fire. (see 2 above)

ad 2/4. My implementation prevents the use of the comma operator, so the point 2 is clearly taken. Any decent optimizer should also eliminate any call to the inline identity function that is only there to prevent inadvertent use of the comma operator or missing to provide an argument. However, my implementation provides no protection against using zero arguments or the comma operator if `NDEBUG` is defined due to the nature of `assert()` being a macro. The empty argument case could be addressed there as well, but I did not attempt that (yet), because it could lead to side effects or make the solution not backward compatible with older version of the standards, where `assert(arg,...)` would require at least 2 arguments, whereas in C++20 it only requires a single argument.

§ 4.6 Do you have implementation experience?

As stated above and can be seen in the appendix, I used an adapted `assert.h` on my system and compiled code with it over several months. I found bugs in my implementation, and I cannot guarantee, that there are no corner cases, I missed. But all bugs that I had during that time, stemmed not from the macro being variadic, but my feeble attempts to detect the cases where a comma operator might sneak in and my brain having forgotten C.

§ 4.7 Why don't you use `bool(__VA_ARGS__)` to prevent comma operator usage?

I did opt for the usage of an identity function, because that approach works both for C++ and C, whereas the suggested remedy `bool(__VA_ARGS__)` would only work for C++. This way, I could prevent implementation divergence between C and C++ of the actual macro replacement. However, because I doubt WG14 will accept the proposal, I prepared WG21 wording using this.

§ 4.8 Why don't you use `assert(arg,__VA_ARGS__)` to prevent zero arguments for the NDEBUG case?

As stated above, the feature is only usable in C++20 for at least one argument macros and I want my system header compile with all versions of C++ and C.

I do not have the resources to do a wider spread analysis of the change, and would appreciate help, if such is required before further consideration.

§ 5 Wording for C++

The change is relative to n4892. I provide the C++ only change version here. For a change that relies on the WG14 draft standard to change as well, see the R0 version of this paper.

In [`assertions.general`] apply the following change (taken from C wording):

1 The header `<cassert>` provides a macro for documenting C++ program assertions and a mechanism for disabling the assertion checks through defining the macro `NDEBUG`.

In [`cassert.syn`] change the macro definition as follows:

```
#define assert( E ... ) see below
```

The contents are the same as the C standard library header <assert.h>, except that a macro named static_assert is not defined.

See also: ISO C 7.2

In [assertions.assert] perform the following changes.

5.0.1 19.3.2 The assert macro [assertions.assert]

1 If NDEBUG is defined as a macro name at the point in the source file where <cassert> is included, the assert macro is defined as

```
#define assert(...) ((void)0)
```

2 Otherwise, the assert macro puts a diagnostic test into programs; it expands to an expression of type void, that when executed evaluates as a subexpression bool(__VA_ARGS__). If that evaluation is false, the assert macro's expression creates a diagnostic containing #__VA_ARGS__ and information on the name of the source file, the source line number, and the name of the enclosing function (such as provided by source_location::current()) on the standard error stream in an implementation-defined format. It then calls abort(). If the argument to assert evaluates to true, there is no further effect.

3 The macro assert is redefined according to the current state of NDEBUG each time that <cassert> is included.

4 An expression assert(E) is a constant subexpression (16.3.6), if

— NDEBUG is defined at the point where assert is last defined or redefined, or

— E contextually converted to bool (7.3) is a constant subexpression that evaluates to the value true.

6 Wording for C

These changes are relative to N2573. If those changes are applied the minimal change for C++ standard proposed in P2264R0 could be used.

In section 7.2 (Diagnostics <assert.h>) change the definition of the assert() macro to use elipsis instead of a single macro parameter:

1 The header <assert.h> defines the assert and static_assert macros and refers to another macro,

```
NDEBUG
```

which is not defined by <assert.h>. If NDEBUG is defined as a macro name at the point in the source file where <assert.h> is included, the assert macro is defined simply as

```
- #define assert(ignore) ((void)0)
+ #define assert(...) ((void)0)
```

The assert macro is redefined according to the current state of NDEBUG each time that <assert.h> is included.

2 The assert macro shall be implemented as a macro with an ellipsis parameter, not as an actual function. If the macro definition is suppressed in order to access an actual function, the behavior is undefined.

In section 7.2.1 (Program Diagnostics) no change is needed. It is included here for easier reference by reviewers.

6.0.1 7.2.1 Program diagnostics

6.0.1.1 7.2.1.1 The assert macro

Synopsis

```
#include <assert.h>
void assert(scalar expression);
```

Description

2 The `assert` macro puts diagnostic tests into programs; it expands to a void expression. When it is executed, if `expression` (which shall have a scalar type) is false (that is, compares equal to 0), the `assert` macro writes information about the particular call that failed (including the text of the argument, the name of the source file, the source line number, and the name of the enclosing function – the latter are respectively the values of the preprocessing macros `__FILE__` and `__LINE__` and of the identifier `__func__`) on the standard error stream in an implementation-defined format.[1] It then calls the `abort` function.

Returns

3 The `assert` macro returns no value.

Forward references: the `abort` function (7.22.4.1).

§ 7 Acknowledgements

Many thanks to Shafik Yaghmour and other Twitterers for inspiring this “janitorial” clean up paper.

Thanks to the reviewers and discussion participants in LEWG, SG22 and WG14.

§ 8 Example implementation (BSD license on MacOS)

The implementation and some simple tests for checking for the non-compilability of calling `assert()` with the wrong number of arguments are in https://github.com/PeterSommerlad/SC22WG21_Papers/tree/master/workspace/p2264_test_for_assert_dotdotdot_on_my_machine.

Here are the key changes. I introduced

```
#ifdef NDEBUG
#define assert(...) ((void)0)
#else

#ifndef CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#define CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#ifdef __cplusplus
#if __cplusplus >= 201103L
 constexpr
#endif
  inline bool __check_single_argument_passed_to_assert(bool b) { return b; }
#else
inline int __check_single_argument_passed_to_assert(int b) { return b; }
// need to have one extern declaration of inline function introduced
// to prevent linker errors. did not patch my libc for that.
extern int __check_single_argument_passed_to_assert(int b);
#endif
#endif /* CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED */

#define assert(...)  \
    ((void) (__check_single_argument_passed_to_assert(__VA_ARGS__) ? ((void)0) : __assert
        (#_VA_ARGS__, __FILE__, __LINE__)))
```

Full file:

```
/*-
 * Copyright (c) 1992, 1993
 *  The Regents of the University of California.  All rights reserved.
 * (c) UNIX System Laboratories, Inc.
 * All or some portions of this file are derived from material licensed
 * to the University of California by American Telephone and Telegraph
 * Co. or Unix System Laboratories, Inc. and are reproduced herein with
 * the permission of UNIX System Laboratories, Inc.
 *
```

```
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 * 3. All advertising materials mentioning features or use of this software
 *    must display the following acknowledgement:
 *  This product includes software developed by the University of
 *  California, Berkeley and its contributors.
 * 4. Neither the name of the University nor the names of its contributors
 *    may be used to endorse or promote products derived from this software
 *    without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED.  IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 *
 *  @(#)assert.h    8.2 (Berkeley) 1/21/94
 * $FreeBSD: src/include/assert.h,v 1.4 2002/03/23 17:24:53 imp Exp $
 */

#include <sys/cdefs.h>
#ifdef __cplusplus
#include <stdlib.h>
#endif /* __cplusplus */

/*
 * Unlike other ANSI header files, <assert.h> may usefully be included
 * multiple times, with and without NDEBUG defined.
 */

#undef assert
#undef __assert

#ifdef NDEBUG
#define assert(...) ((void)0)
#else

#ifndef CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#define CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED
#ifdef __cplusplus
#if __cplusplus >= 201103L
 constexpr
#endif
  inline bool __check_single_argument_passed_to_assert(bool b) { return b; }
#else
inline int __check_single_argument_passed_to_assert(int b) { return b; }
// need to have one extern declaration of inline function introduced
// to prevent linker errors. did not patch my libc for that.
extern int __check_single_argument_passed_to_assert(int b);
#endif
#endif /* CHECK_SINGLE_ASSERT_ARGUMENT_PASSED_TO_ASSERT_DEFINED */

#ifndef __GNUC__

__BEGIN_DECLS
#ifndef __cplusplus
void abort(void) __dead2;
```

```
#endif /* !__cplusplus */
int  printf(const char * __restrict, ...);
__END_DECLS

#define assert(...)  \
    ((void) (__check_single_argument_passed_to_assert(__VA_ARGS__) ? ((void)0) : __assert
        (#_VA_ARGS__, __FILE__, __LINE__)))
#define __assert(e, file, line) \
    ((void)printf ("%s:%u: failed assertion `%s'\n", file, line, e), abort())

#else /* __GNUC__ */

__BEGIN_DECLS
void __assert_rtn(const char *, const char *, int, const char *) __dead2 __disable_tail_calls;
#if defined(__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__) &&
        ((__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__-0) < 1070)
void __eprintf(const char *, const char *, unsigned, const char *) __dead2;
#endif
__END_DECLS

#if defined(__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__) &&
        ((__ENVIRONMENT_MAC_OS_X_VERSION_MIN_REQUIRED__-0) < 1070)
#define __assert(e, file, line) \
    __eprintf ("%s:%u: failed assertion `%s'\n", file, line, e)
#else
/* 8462256: modified __assert_rtn() replaces deprecated __eprintf() */
#define __assert(e, file, line) \
    __assert_rtn ((const char *)-1L, file, line, e)
#endif

#if __DARWIN_UNIX03
#define assert(...) \
    (__builtin_expect(!__check_single_argument_passed_to_assert(__VA_ARGS__), 0) ?
        __assert_rtn(__func__, __FILE__, __LINE__, #__VA_ARGS__) : (void)0)
#else /* !__DARWIN_UNIX03 */
#define assert(...)  \
    (__builtin_expect(!__check_single_argument_passed_to_assert(__VA_ARGS__), 0) ? __assert
        (#__VA_ARGS__, __FILE__, __LINE__) : (void)0)
#endif /* __DARWIN_UNIX03 */

#endif /* __GNUC__ */
#endif /* NDEBUG */

#ifndef _ASSERT_H_
#define _ASSERT_H_

#ifndef __cplusplus
#if defined(__STDC_VERSION__) && __STDC_VERSION__ >= 201112L
#define static_assert _Static_assert
#endif /* __STDC_VERSION__ */
#endif /* !__cplusplus */

#endif /* _ASSERT_H_ */
```

---

1. The message might be of the form: `Assertion failed: _expression_ function _abc_, file _xyz_ line _nnn_.` ↩